Lab Manual

of

Object Oriented Programming

Subject Code: 204ES

Academic Year 2026-27

Name of Student	:	Het Patel
Student ID No	:	25EC412
Batch	:	A
Lab Faculty Name	:	Mayur Sevak

BACHELOR OF ENGINEERING

In

Electronics and Communication



BVM Engineering College

(An Autonomous Institution)

Vallabh Vidhyanagar-388120

BVM ENGINEERING COLLEGE

(An Autonomous Institution)
Electronics and Communication Department
AY: 2026-27, Semester-III

CERTIFICATE

This is to certify that Mr. / Ms. **Het Patel** of Year/Sem. 2nd / III ID No 25EC412 has satisfactorily completed his/her term work in Month of _____ for the term ending in November-2026. Number of practical certified _____ out of _____ in the subject of **204ES: Object Oriented Programming**.

Date: / / 2026

Signature of Teacher



BVM ENGINEERING COLLEGE

(An Autonomous Institution)

Electronics and Communication Department

AY: 2026-27, Semester-III

Object Oriented Programming (204ES)

INDEX

Name: Het Patel **ID No.:** 25EC412

Pr. No.	Title	Page No.	Date	Sign
1.	Write a Program to display basic details of a User like Name, Roll no, Division and Address.	5		
2.	Write a Program to make simple calculator using Switch case.	7		
3.	Write a Program to Convert number of Days into Number of Months and remaining days.	9		
4.	Write a Program to enter marks of 6 subject and find out Total Marks and Percentage.	11		
5.	Write a Program to find whether the given number is even or odd.	13		
6.	Write a Program to find the largest among three numbers.	15		
7.	Write a Program to find whether given number is Armstrong or not.	17		
8.	Write a Program which generates table of a given number.	19		
9.	Write a Program which uses concept of Reference Variable.	21		
10.	Write a Program to swap two numbers using Call by Value method.	23		
11.	Write a Program to swap two numbers using Call by Reference Method.	25		
12.	Write a Program which uses Function as default argument.	27		
13.	Write a Program to calculate factorial of a number using recursion.	29		
14.	Write a Program to take input in an array and print its sum.	31		
15.	Write a Program to read string and print total number of characters.	33		
16.	Write a Program to create a class called 'Student' that has Name, ID, CPI, SPI and Semester as data Member. Also uses Proper member Function for taking and Displaying information.	35		
17.	Write a Program to count number of objects using static variable.	37		

Pr. No.	Title	Page No.	Date	Sign
18.	Define a class to represent a bank account which includes the following members as data member: a) Name of Depositor b) Account Number c) Withdrawal Amount d) Balance. Member Function: a) To assign initial Value b) To Deposit an amount c) To withdraw an amount after checking balance d) To display name and balance. Write the above program for handling n number of account holder using array of object.	39		
19.	Write a Program to demonstrate the use of Constructors and Destructors.	42		
20.	Write a Program which reads the complex number, copy that into other using copy constructor.	44		
21.	Write a Program implement function Overloading for geometric area.	46		
22.	Write a Program for addition of two complex number using Friend Function. (Use constructor function to initialize data member of complex class.)	48		
23.	Write a Program to find the mean value of a given number using friend function.	50		
24.	Write a program to add integer value of two class objects.	52		
25.	Write a Program to do addition of two times objects.	54		
26.	Write a Program to design a class having static member function named showcount() which has the property of displaying the number of object created of the class.	56		
27.	Write a program which uses concept of Unary Operator (++/–) overloading.	58		
28.	Write a program which uses concept of Binary Operator overloading on String.	60		
29.	Write a program to create a Matrix and overload the '+' operator to add two matrix object.	62		
30.	Define two classes polar and rectangle to represent point in polar and rectangle systems. Use conversion routines to convert from one system to the other.	64		
31.	Write a program to implement Single Inheritance.	66		
32.	Write a program to implement Multiple Inheritance.	68		
33.	Write a program to create File with constructor Function.	70		
34.	WAP to demonstrate the use of Template.	72		

Program – 1

Aim:

To write a C++ program to accept and display the basic details of a user (Name, Roll No, Division and Address).

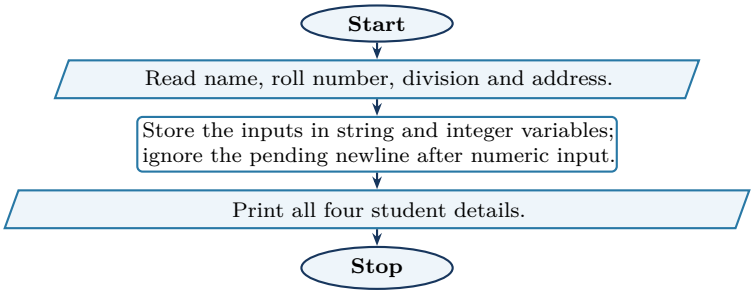
Theory:

C++ provides `cin` and `cout` for reading input and displaying results. The `string` type stores text, while `getline()` reads a complete line including spaces. When formatted input such as `cin » roll` is followed by `getline()`, `cin.ignore()` removes the pending newline so that the next line is read correctly.

Algorithm:

1. Start the program.
2. Declare variables for the name, roll number, division and address.
3. Read the name with `getline()`, then read the roll number with `cin`.
4. Discard the remaining newline using `cin.ignore()`.
5. Read the division and address as complete lines.
6. Display all entered details and terminate.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string name, division, address;
    int roll;
    cout << "Enter Name: ";
    getline(cin, name);
    cout << "Enter Roll No: ";
    cin >> roll;
    cin.ignore();
    cout << "Enter Division: ";
    getline(cin, division);
    cout << "Enter Address: ";
    getline(cin, address);
    cout << "\n------ Student Details ------";
    cout << "\nName     : " << name;
    cout << "\nRoll No  : " << roll;
```

```
    cout << "\nDivision : " << division;
    cout << "\nAddress  : " << address;
    return 0;
}
```

Output:

```
Enter Name: Aarav Mehta
Enter Roll No: 27
Enter Division: EC-A
Enter Address: Vadodara

------ Student Details ------
Name     : Aarav Mehta
Roll No  : 27
Division : EC-A
Address  : Vadodara
```

Conclusion:

In this practical, a C++ program was written to accept a user's Name, Roll No, Division and Address and display them in a neat format. The use of `getline()` allowed the name, division and address to be read along with spaces, while `cin.ignore()` removed the leftover newline after reading the roll number so that the next `getline()` did not get skipped. The practical clarified how `cin` and `cout` handle formatted input and output and how the input buffer behaves when numeric and string inputs are mixed.

Program – 2

Aim:

To write a C++ program to perform simple arithmetic operations (addition, subtraction, multiplication, division and modulus) using the switch-case statement.

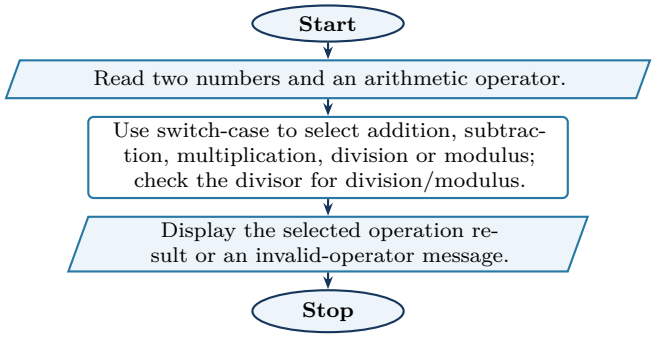
Theory:

The `switch` statement selects one operation from several fixed choices based on the operator entered. The `break` statement prevents execution from continuing into another case, and `default` handles an unsupported operator. Division and modulus require a non-zero divisor; casting the numerator to `float` allows division to produce a fractional result.

Algorithm:

1. Start the program and declare two integers and an operator character.
2. Input both numbers and the required arithmetic operator.
3. Use `switch` to select addition, subtraction, multiplication, division or modulus.
4. Before division or modulus, check whether the second number is zero.
5. Display the calculated result or the appropriate error message.
6. Terminate the program.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num1, num2;
    char op;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "Enter operator (+, -, *, /, %): ";
    cin >> op;
    switch (op)
    {
        case '+':
            cout << "Addition = " << num1 + num2;
            break;
        case '-':
```

```
            cout << "Subtraction = " << num1 - num2;
            break;
        case '*':
            cout << "Multiplication = " << num1 * num2;
            break;
        case '/':
            if (num2 != 0)
                cout << "Division = " << (float)num1 / num2;
            else
                cout << "Division by zero is not allowed.";
            break;
        case '%':
            if (num2 != 0)
                cout << "Modulus = " << num1 % num2;
            else
                cout << "Modulus by zero is not allowed.";
            break;
        default:
            cout << "Invalid Operator!";
    }
    return 0;
}
```

Output 1:

```
Enter first number: 36
Enter second number: 14
Enter operator (+, -, *, /, %): +
Addition = 50
```

Output 2:

```
Enter first number: 81
Enter second number: 9
Enter operator (+, -, *, /, %): /
Division = 9
```

Output 3:

```
Enter first number: 29
Enter second number: 6
Enter operator (+, -, *, /, %): %
Modulus = 5
```

Conclusion:

A simple calculator was built using the `switch-case` statement, which selects the operation according to the operator entered. The sample runs gave correct results (20 + 10 = 30, 25 / 5 = 5 and 15 % 4 = 3). The program also checks for a zero divisor before division and modulus and uses the `default` case for invalid operators, showing how `switch` gives a cleaner alternative to a long `if-else` ladder when one variable is compared against many constant values.

Program – 3

Aim:

To write a C++ program to convert a given number of days into months and remaining days.

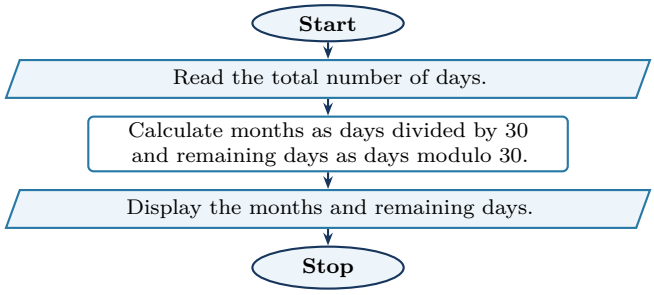
Theory:

Integer division by 30 gives the number of complete 30-day months, and the modulus operator gives the remaining days. This calculation uses the practical's simplifying assumption that every month contains exactly 30 days; it is not a calendar-date conversion.

Algorithm:

1. Start the program.
2. Read the total number of days.
3. Divide the total by 30 to obtain complete months.
4. Find the remaining days using the modulus operator with 30.
5. Display the months and remaining days.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int days, months, remainingDays;
    cout << "Enter the number of days: ";
    cin >> days;
    months = days / 30;
    remainingDays = days % 30;
    cout << "\nEquivalent Time:";
    cout << "\nMonths = " << months;
    cout << "\nRemaining Days = " << remainingDays;
    return 0;
}
```

Output 1:

```
Enter the number of days: 94

Equivalent Time:
Months = 3
Remaining Days = 4
```

Output 2:

```
Enter the number of days: 67

Equivalent Time:
Months = 2
Remaining Days = 7
```

Output 3:

```
Enter the number of days: 155

Equivalent Time:
Months = 5
Remaining Days = 5
```

Conclusion:

The program converted a number of days into months and remaining days by assuming 30 days per month. Integer division (`/`) gave the number of complete months and the modulus operator (`%`) gave the leftover days, so 94 days produced 3 months and 4 days, 67 days produced 2 months and 7 days, and 155 days produced 5 months and 5 days. This practical showed the different roles of the division and modulus operators on integer operands.

Program – 4

Aim:

To write a C++ program to enter the marks of six subjects and find the total marks and percentage.

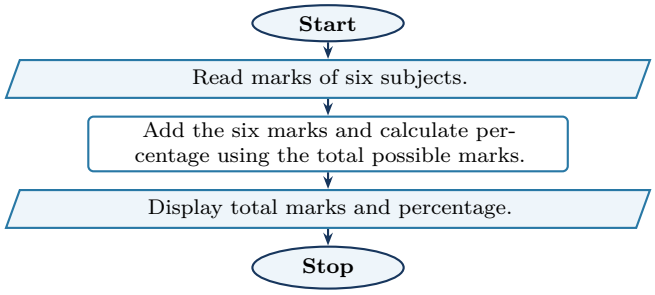
Theory:

The total marks are obtained by adding the marks of all six subjects. Percentage is calculated by dividing the total by the maximum possible marks and multiplying by 100. Using a floating-point divisor such as `6.0` prevents integer division from discarding the fractional part of the result.

Algorithm:

1. Start the program.
2. Declare variables for the six subject marks, total and percentage.
3. Input the marks for all six subjects.
4. Add the six marks to calculate the total.
5. Calculate percentage using the total and maximum marks per subject.
6. Display the total and percentage, then stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int sub1, sub2, sub3, sub4, sub5, sub6;
    int total;
    float percentage;
    cout << "Enter marks of Subject 1: ";
    cin >> sub1;
    cout << "Enter marks of Subject 2: ";
    cin >> sub2;
    cout << "Enter marks of Subject 3: ";
    cin >> sub3;
    cout << "Enter marks of Subject 4: ";
    cin >> sub4;
    cout << "Enter marks of Subject 5: ";
    cin >> sub5;
    cout << "Enter marks of Subject 6: ";
    cin >> sub6;
    total = sub1 + sub2 + sub3 + sub4 + sub5 + sub6;
    percentage = total / 6.0;
    cout << "\nTotal Marks = " << total;
```

```
    cout << "\nPercentage = " << percentage << "%";
    return 0;
}
```

Output:

```
Enter marks of Subject 1: 91
Enter marks of Subject 2: 84
Enter marks of Subject 3: 77
Enter marks of Subject 4: 89
Enter marks of Subject 5: 95
Enter marks of Subject 6: 80

Total Marks = 516
Percentage = 86%
```

Conclusion:

The total marks and percentage of six subjects were calculated from the entered marks; for the sample input the total was 500 and the percentage was 83.3333%. Dividing the total by `6.0` instead of `6` forces floating-point division, so the fractional part is not lost. The practical demonstrated the use of multiple variables, arithmetic expressions and the need to choose the correct data type (`int` for total, `float` for percentage) for the result.

Program – 5

Aim:

To write a C++ program to find whether a given number is even or odd.

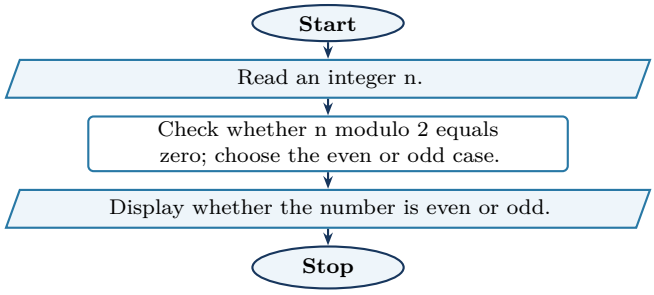
Theory:

An integer is even if it is divisible by 2 without a remainder; otherwise, it is odd. The modulus operator `%` returns the remainder, and an `if-else` statement chooses which message to display based on whether that remainder is zero.

Algorithm:

1. Start the program.
2. Read an integer from the user.
3. Calculate the remainder when the integer is divided by 2.
4. If the remainder is zero, display that the number is even; otherwise display that it is odd.
5. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num % 2 == 0)
        cout << num << " is an Even number.";
    else
        cout << num << " is an Odd number.";
    return 0;
}
```

Output 1:

```
Enter a number: 24
24 is an Even number.
```

Output 2:

```
Enter a number: 13
13 is an Odd number.
```

Conclusion:

The program checked whether a number is even or odd by testing the remainder of division by 2 with the modulus operator. The entered values 12 and 7 were correctly reported as even and odd respectively. This practical demonstrated the use of the `if-else` selection statement together with a relational expression, which is the basis of most decision-making in C++ programs.

Program – 6

Aim:

To write a C++ program to find the largest among three numbers.

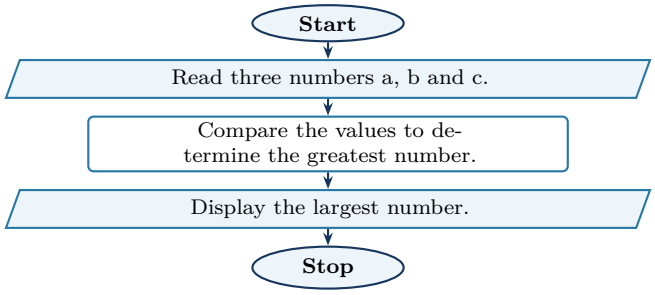
Theory:

To identify the largest of three numbers, each candidate is compared with the other two. Relational operators form the comparisons, while the logical AND operator `&&` combines the conditions. Using `>=` also allows the comparisons to succeed when values are equal.

Algorithm:

1. Start the program.
2. Read three numbers.
3. Compare the first number with the second and third.
4. If it is not the largest, compare the second number with the first and third.
5. If neither condition is true, select the third number.
6. Display the largest value and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;
    if (a >= b && a >= c)
        cout << "Largest number is: " << a;
    else if (b >= a && b >= c)
        cout << "Largest number is: " << b;
    else
        cout << "Largest number is: " << c;
    return 0;
}
```

Output:

```
Enter three numbers: 42 17 36
Largest number is: 42
```

Conclusion:

The largest of three numbers was found using an `if-else if` ladder in which each number is compared with the other two using the relational operator `>=` and the logical AND operator `&&`. For the input 42, 17 and 36 the program correctly displayed 42. Using `>=` ensures the program also works when two or more numbers are equal, and the practical illustrated how compound conditions are formed with logical operators.

Program – 7

Aim:

To write a C++ program to check whether a given number is an Armstrong number or not.

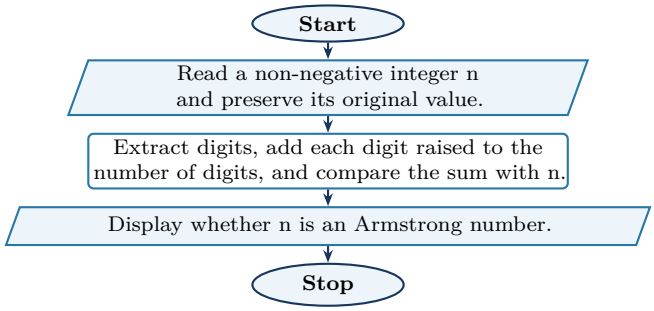
Theory:

An Armstrong number is a non-negative integer equal to the sum of each of its digits raised to the power of the number of digits. The modulus operator extracts the last digit, and integer division removes that digit. The `pow()` function from `<cmath>` calculates powers; its result is rounded before being added to the integer sum.

Algorithm:

1. Start the program and read the number.
2. Count the number of digits in the input.
3. Copy the number into a temporary variable and initialise the sum to zero.
4. Repeatedly extract the last digit, raise it to the digit count, add it to the sum and remove the digit.
5. Compare the calculated sum with the original number.
6. Display whether the number is an Armstrong number and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
    int num, originalNum, remainder, digits = 0;
    int result = 0;
    cout << "Enter a number: ";
    cin >> num;
    originalNum = num;
    // Count the number of digits
    for (; originalNum != 0; originalNum /= 10)
        digits++;
    originalNum = num;
    // Sum of digits raised to the power of number of digits
    for (; originalNum != 0; originalNum /= 10)
    {
        remainder = originalNum % 10;
        result += (int)round(pow(remainder, digits));
```

```
    }
    if (result == num)
        cout << num << " is an Armstrong Number.";
    else
        cout << num << " is not an Armstrong Number.";
    return 0;
}
```

Output 1:

```
Enter a number: 407
407 is an Armstrong Number.
```

Output 2:

```
Enter a number: 246
246 is not an Armstrong Number.
```

Conclusion:

The program determined whether a number is an Armstrong number by counting its digits and adding each digit raised to the power of the digit count. The number 371 was identified as an Armstrong number because 3^3 + 7^3 + 1^3 = 371, while 246 was correctly rejected. The practical combined loops, the modulus and division operators and the `pow()` function of `<cmath>`, and showed the need to round the floating-point result of `pow()` before storing it in an integer.

Program – 8

Aim:

To write a C++ program to generate the multiplication table of a given number.

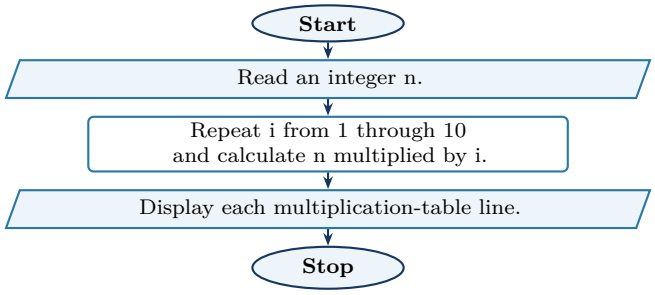
Theory:

A `for` loop is suited to a task with a known number of repetitions. Here, the loop variable runs from 1 through 10, and each iteration multiplies it by the input number to generate one row of the multiplication table.

Algorithm:

1. Start the program.
2. Read the number whose table is required.
3. Initialise a loop counter to 1.
4. Repeat while the counter is at most 10: multiply the input number by the counter and display the equation.
5. Increment the counter after each row.
6. Stop after the tenth row.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << "\nMultiplication Table of " << num << ":\n";
    for (int i = 1; i <= 10; i++)
        cout << num << " x " << i << " = " << num * i << endl;
    return 0;
}
```

Output:

```
Enter a number: 9

Multiplication Table of 9:
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

Conclusion:

The multiplication table of the entered number was generated with a `for` loop that runs from 1 to 10; for the input 9 the program printed the table from 9 x 1 = 9 up to 9 x 10 = 90. The practical showed how a counted loop repeats a block of statements and how the loop variable can be used directly inside the output expression, avoiding ten separate statements.

Program – 9

Aim:

To write a C++ program that demonstrates the concept of a reference variable.

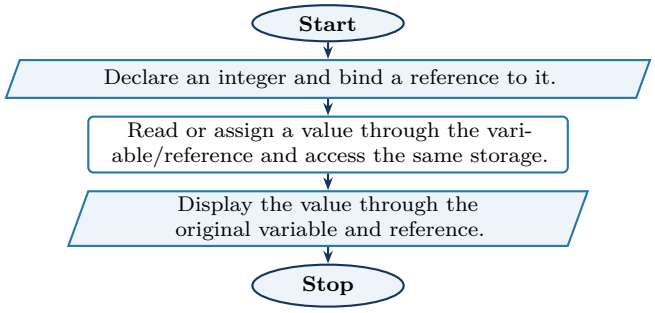
Theory:

A reference is an alias for an existing variable, not an independent copy. It must be initialised when declared, and assignments made through either name affect the same underlying object. This behaviour is useful for passing variables to functions by reference.

Algorithm:

1. Start the program.
2. Declare and initialise an integer variable.
3. Declare a reference that refers to that integer.
4. Display the original variable and its reference.
5. Modify the value through the reference and display the original variable again.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    int &ref = num;   // Reference variable
    cout << "\nOriginal value of num = " << num << endl;
    cout << "Value using reference variable = " << ref << endl;
    ref = ref + 10;   // Modify through reference
    cout << "\nAfter modifying through reference variable:" << endl;
    cout << "Value of num = " << num << endl;
    cout << "Value of ref = " << ref << endl;
    return 0;
}
```

Output:

```
Enter a number: 42

Original value of num = 42
Value using reference variable = 42

After modifying through reference variable:
Value of num = 52
Value of ref = 52
```

Conclusion:

A reference variable `ref` was created as an alias of the variable `num`, and the program showed that both names refer to the same memory location. When 10 was added through the reference, the value of `num` also changed from 42 to 52. This practical explained that a reference must be initialised when declared and that it avoids copying, which is the idea behind call by reference and efficient parameter passing.

Program – 10

Aim:

To write a C++ program to swap two numbers using the call by value method.

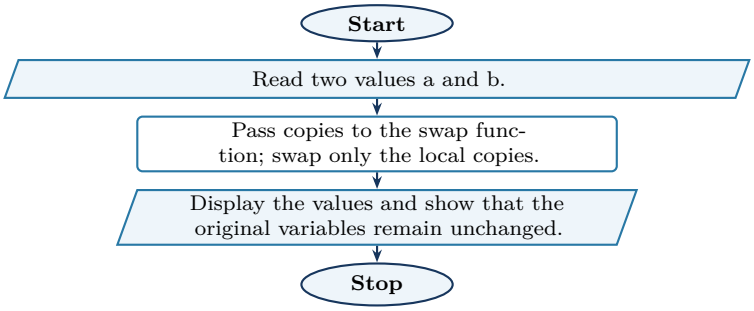
Theory:

In call by value, a function receives copies of the supplied arguments. Swapping its local parameter variables changes only those copies, so the variables in the calling function remain unchanged after the call.

Algorithm:

1. Start the program.
2. Declare and initialise two variables in `main()`.
3. Call the swap function with the two variables by value.
4. Inside the function, exchange the copied parameter values using a temporary variable.
5. Display the values inside the function and then display the original variables in `main()`.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Function using Call by Value
void swapValues(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    cout << "\nInside swapValues() function:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}
int main()
{
    int x, y;
    cout << "Enter two numbers: ";
    cin >> x >> y;
    cout << "\nBefore swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    swapValues(x, y);
    cout << "\nAfter calling swapValues() function:" << endl;
```

```
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

```
Enter two numbers: 18 35

Before swapping:
x = 18, y = 35

Inside swapValues() function:
a = 35, b = 18

After calling swapValues() function:
x = 18, y = 35
```

Conclusion:

The swap function in this program received copies of the arguments, so the values were exchanged only inside the function (a = 20, b = 10), while the original variables in `main()` remained x = 10 and y = 20. This confirmed that, in call by value, changes made to the formal parameters never affect the actual arguments, and that a function working on copies cannot be used to modify the caller's variables.

Program – 11

Aim:

To write a C++ program to swap two numbers using the call by reference method.

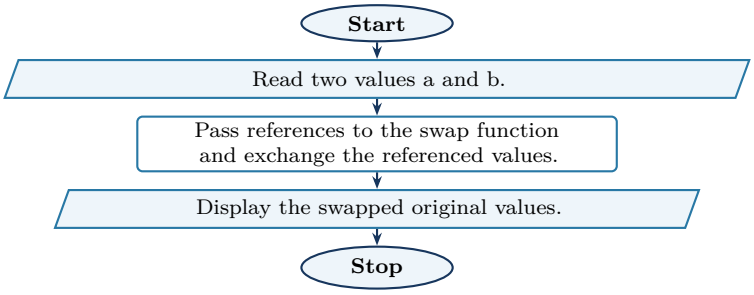
Theory:

Call by reference allows function parameters to refer directly to the caller's variables. Reference parameters, written with `&`, do not create separate values, so assignments within the function affect the original arguments.

Algorithm:

1. Start the program.
2. Declare and initialise two variables.
3. Pass both variables to the swap function through reference parameters.
4. Exchange the values using a temporary variable.
5. Display the values after the function call.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Function using Call by Reference
void swapNumbers(int &a, int &b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
int main()
{
    int num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "\nBefore Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    swapNumbers(num1, num2);
    cout << "\nAfter Swapping:" << endl;
```

```
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    return 0;
}
```

Output:

```
Enter first number: 64
Enter second number: 29

Before Swapping:
First Number = 64
Second Number = 29

After Swapping:
First Number = 29
Second Number = 64
```

Conclusion:

In this program the parameters of `swapNumbers()` were declared as references (`int &a`, `int &b`), so the function worked directly on the original variables of `main()`. The values 25 and 40 were exchanged successfully and the change was visible after the function returned. Compared with call by value, call by reference modifies the actual arguments and avoids making copies, which makes it suitable for functions that must change their arguments.

Program – 12

Aim:

To write a C++ program that uses a function with default arguments.

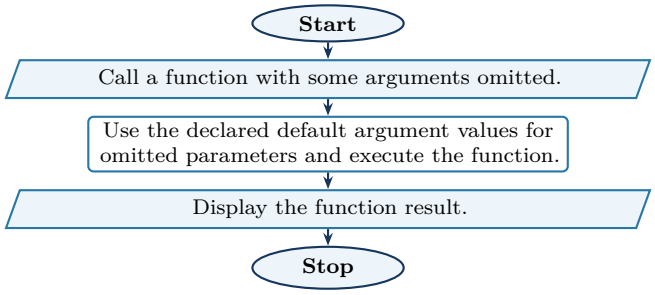
Theory:

Default arguments provide values for parameters omitted in a function call. When arguments are supplied from left to right, the given values replace the corresponding defaults and any omitted trailing parameters retain their defaults. Default values are specified in the declaration/definition from right to left.

Algorithm:

1. Start the program.
2. Define the function with default values for its parameters.
3. Call the function with no arguments and display the returned result.
4. Call it with one argument so the remaining parameter uses its default.
5. Call it with both arguments supplied and display the result.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Function with default arguments
int add(int a = 10, int b = 20)
{
    return a + b;
}
int main()
{
    cout << "add()       = " << add() << endl;       // Uses both defaults
    cout << "add(5)      = " << add(5) << endl;      // a = 5, b = 20
    cout << "add(5, 15) = " << add(5, 15) << endl; // a = 5, b = 15
    return 0;
}
```

Output:

```
add()       = 30
add(5)      = 25
add(5, 15) = 20
```

Conclusion:

The function `add()` was defined with default values for both parameters, and it behaved according to the number of arguments supplied: `add()` returned 30 using both defaults, `add(5)` returned 25 using the default for the second argument, and `add(5, 15)` returned 20 using the values passed. This practical showed that default arguments reduce the need for multiple overloaded functions and must be specified from right to left in the parameter list.

Program – 13

Aim:

To write a C++ program to calculate the factorial of a number using recursion.

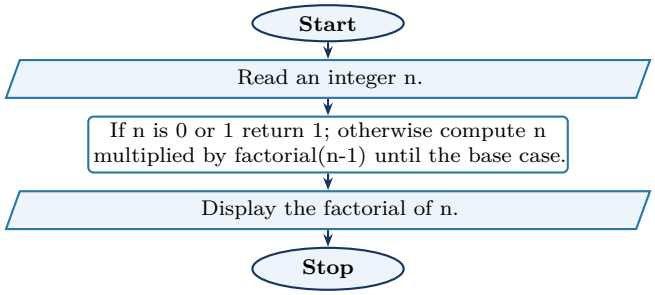
Theory:

Recursion occurs when a function calls itself to solve a smaller version of the same problem. Factorial is defined as $n! = n(n-1)!$, with $0! = 1$ and $1! = 1$ as base cases. A base case is essential to end recursive calls; the program also checks for negative input, for which factorial is not defined here.

Algorithm:

1. Start the program and read an integer.
2. If the input is negative, display the invalid-input message.
3. Otherwise call the recursive factorial function.
4. Return 1 when the argument is 0 or 1.
5. For larger arguments, return the argument multiplied by the factorial of one less.
6. Display the result and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Recursive function to calculate factorial
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num < 0)
        cout << "Factorial of a negative number is not defined.";
    else
        cout << "Factorial of " << num << " = " << factorial(num);
    return 0;
}
```

Output:

```
Enter a number: 6
Factorial of 6 = 720
```

Conclusion:

The factorial of a number was computed by a recursive function that calls itself with `n - 1` until the base case of 0 or 1 is reached; for the input 6 the program displayed 720. The practical showed how a problem can be broken into smaller identical problems, the importance of a base case to stop the recursion, and how the program handles an invalid (negative) input safely.

Program – 14

Aim:

To write a C++ program to take input in an array and print the sum of its elements.

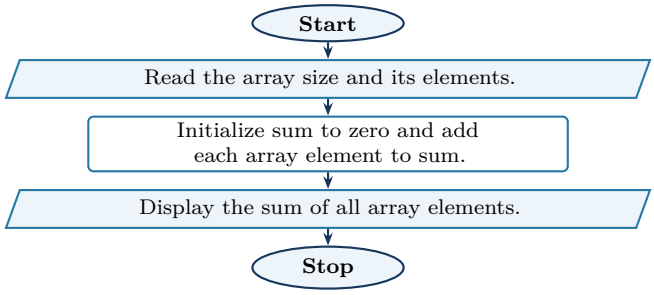
Theory:

An array stores multiple values of the same data type under one name. Individual elements are accessed using zero-based indices, and a loop can visit each element in order. A running total is updated by adding each entered value.

Algorithm:

1. Start the program.
2. Declare an array and a variable for the sum.
3. Read the required array elements from the user.
4. Initialise the sum to zero.
5. Traverse the array with a loop and add each element to the sum.
6. Display the total and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int arr[100], n, sum = 0;
    cout << "Enter the number of elements: ";
    cin >> n;
    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    // Calculate sum of array elements
    for (int i = 0; i < n; i++)
        sum = sum + arr[i];
    cout << "Sum of array elements = " << sum << endl;
    return 0;
}
```

Output:

```
Enter the number of elements: 4
Enter 4 elements:
12
18
24
36
Sum of array elements = 90
```

Conclusion:

The program stored the elements entered by the user in an array and added them with a loop; the four values 12, 18, 24 and 36 gave a sum of 90. The practical demonstrated how an array holds several values of the same type under one name, how elements are accessed by index starting from 0, and how loops are used to traverse an array for input and processing.

Program – 15

Aim:

To write a C++ program to read a string and print the total number of characters.

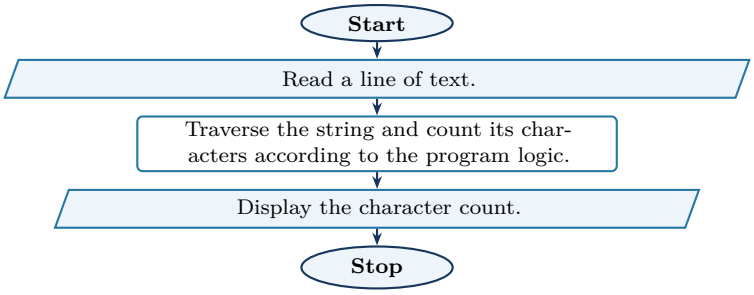
Theory:

The C++ `string` class represents text and provides the `length()` member function to report the number of characters. Spaces count as characters. The `getline()` function reads the full input line, rather than stopping at the first whitespace.

Algorithm:

1. Start the program.
2. Declare a string variable.
3. Read a complete line of text using `getline()`.
4. Call `length()` to obtain the character count.
5. Display the string length and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "Total number of characters = " << str.length() << endl;
    return 0;
}
```

Output:

```
Enter a string: Object Oriented Lab
Total number of characters = 19
```

Conclusion:

The program read a complete line of text using `getline()` and found its length with the `length()` member function of the `string` class; the string "Object Oriented Lab" gave a count of 19, which includes the space. The practical showed the advantage of the `string` class over character arrays, since the length is available directly without writing a counting loop.

Program – 16

Aim:

To write a C++ program to create a class `Student` with Name, ID, CPI, SPI and Semester as data members, and member functions to read and display the information.

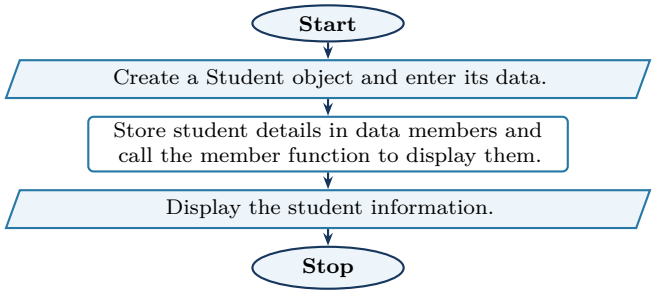
Theory:

A class is a user-defined type that groups data members and member functions. An object is an instance of that class. Private members are hidden from direct access outside the class, while public member functions provide a controlled interface for reading and displaying the student's information.

Algorithm:

1. Start the program.
2. Define the `Student` class with data members and public input/display functions.
3. Create a `Student` object.
4. Call the input function to read the student's details.
5. Call the display function to print the stored details.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
    string name;
    int id;
    float cpi, spi;
    int semester;
public:
    // Member function to accept student details
    void getData()
    {
        cout << "Enter Student Name: ";
        getline(cin, name);
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter CPI: ";
        cin >> cpi;
        cout << "Enter SPI: ";
```

```
        cin >> spi;
        cout << "Enter Semester: ";
        cin >> semester;
    }
    // Member function to display student details
    void displayData()
    {
        cout << "\n----- Student Information -----" << endl;
        cout << "Name     : " << name << endl;
        cout << "ID       : " << id << endl;
        cout << "CPI      : " << cpi << endl;
        cout << "SPI      : " << spi << endl;
        cout << "Semester : " << semester << endl;
    }
};
int main()
{
    Student s;
    s.getData();
    s.displayData();
    return 0;
}
```

Output:

```
Enter Student Name: Diya Shah
Enter Student ID: 214
Enter CPI: 8.42
Enter SPI: 9.25
Enter Semester: 4

----- Student Information -----
Name     : Diya Shah
ID       : 214
CPI      : 8.42
SPI      : 9.25
Semester : 4
```

Conclusion:

A class `Student` was created with private data members (Name, ID, CPI, SPI and Semester) and public member functions to read and display them. An object of the class was used to take input and show the stored information. The practical illustrated the basic object-oriented ideas of encapsulation and data hiding, in which data is bound together with the functions that operate on it and is accessed only through the public interface.

Program – 17

Aim:

To write a C++ program to count the number of objects created using a static variable.

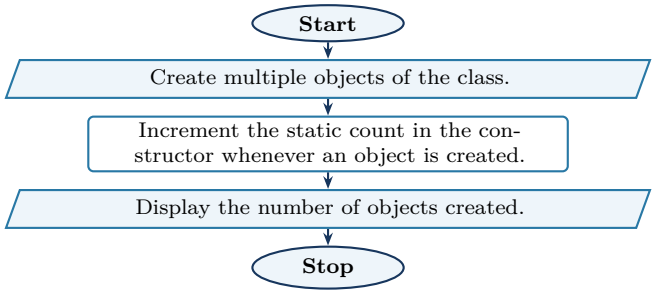
Theory:

A static data member has one shared instance for the entire class rather than a separate copy in every object. A static member function can be called using the class name and can directly access static members. Defining the static data member outside the class provides its storage.

Algorithm:

1. Start the program.
2. Declare a static counter in the class and define it outside the class.
3. Increment the counter whenever the constructor creates an object.
4. Create three objects.
5. Call the static display function to show the shared count.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Student
{
private:
    static int count;   // Static data member
public:
    // Constructor
    Student()
    {
        count++;
        cout << "Object " << count << " created." << endl;
    }
    // Static member function
    static void displayCount()
    {
        cout << "\nTotal number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Student::count = 0;
int main()
```

```
{
    Student s1;
    Student s2;
    Student s3;
    Student::displayCount();
    return 0;
}
```

Output:

```
Object 1 created.
Object 2 created.
Object 3 created.

Total number of objects created = 3
```

Conclusion:

A static data member `count` was shared by all the objects of the class and was incremented in the constructor every time an object was created, so after three objects were created its value was 3. The static member function `displayCount()` was called using the class name without any object. This practical showed that a static member belongs to the class rather than to an individual object and is useful for keeping class-level information such as an object counter.

Program – 18

Aim:

To write a C++ program for a bank account class with the data members name of depositor, account number, withdrawal amount and balance, and the member functions to assign initial values, deposit an amount, withdraw an amount after checking the balance, and display the name and balance, and to handle n account holders using an array of objects.

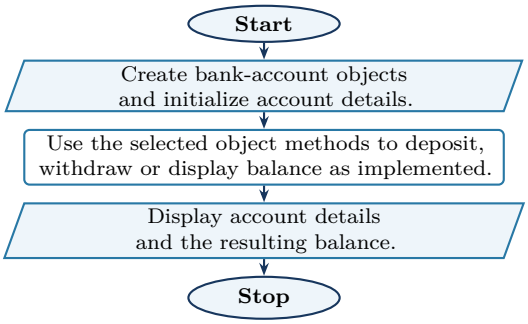
Theory:

An array of objects stores several instances of the same class. Each object keeps its own account holder details and balance, while member functions perform operations on that account. A conditional check can prevent a withdrawal when the requested amount exceeds the available balance.

Algorithm:

1. Start the program.
2. Define the bank-account class and its data members and operations.
3. Create and initialise an array of account objects.
4. For each account, display details and perform the specified deposit or withdrawal.
5. Before a withdrawal, compare the requested amount with the current balance and report insufficient funds if necessary.
6. Display the resulting account balances and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
class BankAccount
{
private:
    string name;
    int accountNo;
    float balance;
    float withdrawAmount;
public:
    // Assign initial values
    void assign()
    {
        cout << "Enter Depositor Name: ";
        cin.ignore();
```

```
        getline(cin, name);
        cout << "Enter Account Number: ";
        cin >> accountNo;
        cout << "Enter Initial Balance: ";
        cin >> balance;
    }
    // Deposit an amount
    void deposit()
    {
        float amount;
        cout << "Enter amount to deposit: ";
        cin >> amount;
        balance = balance + amount;
        cout << "Amount Deposited Successfully." << endl;
    }
    // Withdraw an amount after checking balance
    void withdraw()
    {
        cout << "Enter amount to withdraw: ";
        cin >> withdrawAmount;
        if (withdrawAmount <= balance)
        {
            balance = balance - withdrawAmount;
            cout << "Withdrawal Successful." << endl;
        }
        else
            cout << "Insufficient Balance!" << endl;
    }
    // Display name and balance
    void display()
    {
        cout << "\n-----------------------------" << endl;
        cout << "Depositor Name : " << name << endl;
        cout << "Account Number : " << accountNo << endl;
        cout << "Balance        : " << balance << endl;
    }
};
int main()
{
    int n;
    cout << "Enter Number of Account Holders: ";
    cin >> n;
    BankAccount *acc = new BankAccount[n];   // Array of objects
    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter Details of Account Holder " << i + 1 << endl;
        acc[i].assign();
    }
    for (int i = 0; i < n; i++)
    {
        cout << "\nTransactions for Account Holder " << i + 1 << endl;
        acc[i].deposit();
        acc[i].withdraw();
    }
    cout << "\n\n----- Account Details -----";
    for (int i = 0; i < n; i++)
        acc[i].display();
    delete[] acc;
    return 0;
}
```

Output:

```
Enter Number of Account Holders: 2

Enter Details of Account Holder 1
Enter Depositor Name: Karan Desai
Enter Account Number: 2105
Enter Initial Balance: 6500

Enter Details of Account Holder 2
Enter Depositor Name: Riya Joshi
Enter Account Number: 2106
Enter Initial Balance: 4200

Transactions for Account Holder 1
Enter amount to deposit: 1500
Amount Deposited Successfully.
Enter amount to withdraw: 2500
Withdrawal Successful.

Transactions for Account Holder 2
Enter amount to deposit: 800
Amount Deposited Successfully.
Enter amount to withdraw: 6000
Insufficient Balance!


----- Account Details -----
Depositor Name : Karan Desai
Account Number : 2105
Balance        : 5500

Depositor Name : Riya Joshi
Account Number : 2106
Balance        : 5000
```

Conclusion:

The bank account class was used to handle several account holders at once by creating an array of objects. For each account the program assigned the initial values, deposited an amount and allowed a withdrawal only if the balance was sufficient; Karan Desai's account ended with a balance of 5500, and Riya Joshi's withdrawal of 6000 was rejected with an "Insufficient Balance" message so her balance remained 5000. The practical showed how arrays of objects, member functions and conditional checks work together to model a real-world situation.

Program – 19

Aim:

To write a C++ program to demonstrate the use of constructors and destructors.

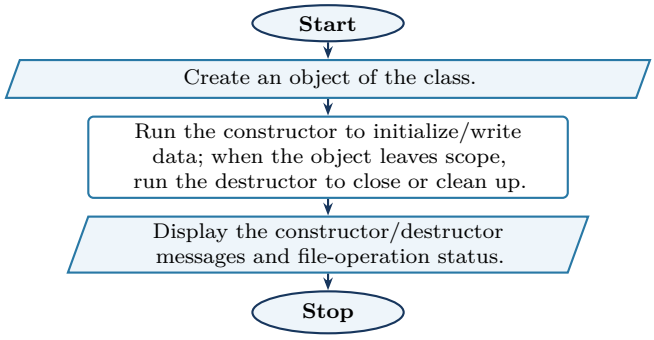
Theory:

A constructor is a special member function invoked automatically when an object is created; it commonly initialises the object. A destructor is invoked automatically when the object is destroyed or leaves scope. Their execution order helps manage an object's lifetime and associated resources.

Algorithm:

1. Start the program.
2. Define a class with a constructor, a display member function and a destructor.
3. Create an object so that the constructor runs.
4. Call the display function.
5. Leave the object's scope so that the destructor runs.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Student
{
public:
    // Constructor
    Student()
    {
        cout << "Constructor Called: Object Created." << endl;
    }
    // Destructor
    ~Student()
    {
        cout << "Destructor Called: Object Destroyed." << endl;
    }
    void display()
    {
        cout << "Welcome to C++ Programming!" << endl;
    }
};
```

```
int main()
{
    Student s1;     // Constructor is called automatically
    s1.display();
    return 0;       // Destructor is called automatically
}
```

Output:

```
Constructor Called: Object Created.
Welcome to C++ Programming!
Destructor Called: Object Destroyed.
```

Conclusion:

The program showed that a constructor is called automatically when an object is created and a destructor is called automatically when the object goes out of scope. The output appeared in the order "Constructor Called", the message from `display()`, and then "Destructor Called". This practical explained how constructors are used to initialise objects and destructors to release resources, and how the lifetime of an object controls when each is executed.

Program – 20

Aim:

To write a C++ program to read a complex number and copy it into another using a copy constructor.

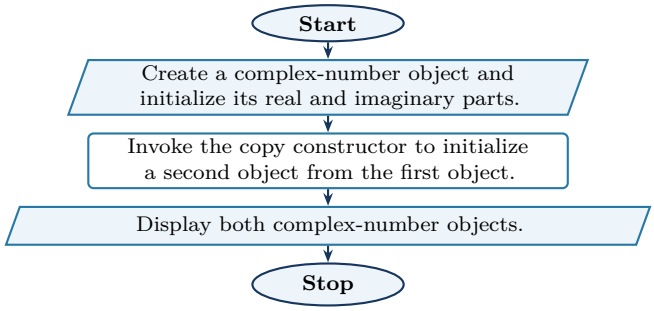
Theory:

A copy constructor initialises a new object from an existing object of the same class. It commonly accepts a constant reference to the source object and copies its data members. This avoids unnecessary copying of the source object itself and is called during copy initialisation.

Algorithm:

1. Start the program.
2. Define the `Complex` class with real and imaginary data members.
3. Implement a copy constructor that copies both members from an existing object.
4. Read the complex number and initialise the first object.
5. Create a second object by copying the first object.
6. Display both objects and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Parameterized constructor
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Copy constructor
    Complex(const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }
    void display()
    {
        cout << real;
```

```
        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";
        cout << endl;
    }
};
int main()
{
    float real, imag;
    cout << "Enter real part: ";
    cin >> real;
    cout << "Enter imaginary part: ";
    cin >> imag;
    Complex c1(real, imag);
    Complex c2(c1);         // Copy c1 into c2 using copy constructor
    cout << "\nOriginal Complex Number: ";
    c1.display();
    cout << "Copied Complex Number:   ";
    c2.display();
    return 0;
}
```

Output:

```
Enter real part: 8
Enter imaginary part: -3

Original Complex Number: 8 - 3i
Copied Complex Number:   8 - 3i
```

Conclusion:

The program read a complex number, created the object `c1` and then created `c2` from it using a copy constructor; both objects displayed the same number, 8 - 3i. The practical explained that a copy constructor takes a reference to an existing object of the same class (`const Complex &c`) and copies its data members, and that it is invoked when a new object is initialised from an existing one.

Program – 21

Aim:

To write a C++ program to implement function overloading for finding geometric areas.

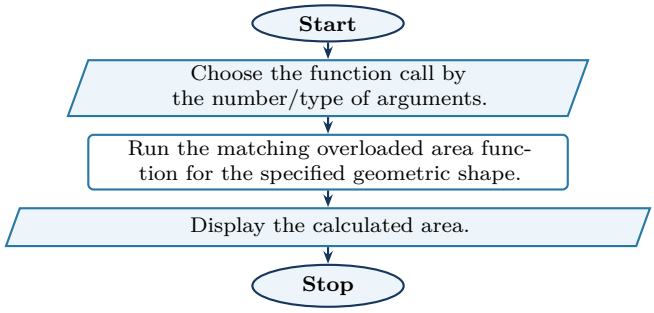
Theory:

Function overloading allows several functions to share a name when their parameter lists differ. The compiler selects the appropriate overload at compile time based on the number and types of arguments. The program uses this feature to calculate areas for different geometric shapes.

Algorithm:

1. Start the program.
2. Define overloaded `area()` functions for a circle, rectangle and triangle using different parameter lists.
3. Call the circle overload with its radius.
4. Call the rectangle overload with its length and width.
5. Call the triangle overload with its base and height.
6. Display each calculated area and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Area of circle
float area(float radius)
{
    return 3.14159 * radius * radius;
}
// Area of rectangle
float area(float length, float breadth)
{
    return length * breadth;
}
// Area of triangle (extra int parameter distinguishes this overload)
float area(float base, float height, int)
{
    return 0.5 * base * height;
}
int main()
{
    float r, l, b, base, h;
```

```
    cout << "Enter radius of circle: ";
    cin >> r;
    cout << "Area of Circle = " << area(r) << endl;
    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;
    cout << "Area of Rectangle = " << area(l, b) << endl;
    cout << "\nEnter base and height of triangle: ";
    cin >> base >> h;
    cout << "Area of Triangle = " << area(base, h, 0) << endl;
    return 0;
}
```

Output:

```
Enter radius of circle: 7
Area of Circle = 153.938

Enter length and breadth of rectangle: 12 4
Area of Rectangle = 48

Enter base and height of triangle: 9 8
Area of Triangle = 36
```

Conclusion:

Three functions with the same name `area()` but different parameter lists were defined for a circle, a rectangle and a triangle, and the compiler selected the correct one from the arguments in each call. The areas obtained were 78.5397 for a circle of radius 5, 60 for a 10 by 6 rectangle and 20 for a triangle of base 8 and height 5. The practical demonstrated function overloading, a form of compile-time polymorphism that allows one function name to be used for related operations.

Program – 22

Aim:

To write a C++ program for the addition of two complex numbers using a friend function.

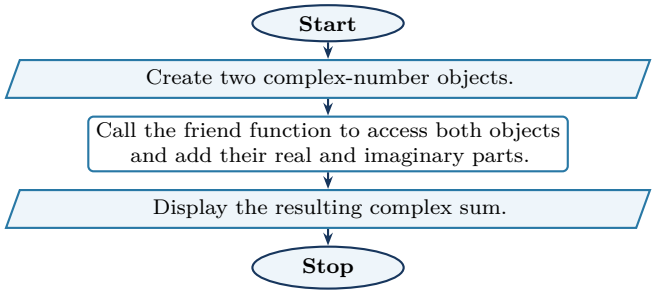
Theory:

A friend function is not a member of a class, but a class can grant it permission to access private and protected members. It is declared with the `friend` keyword inside the class and receives objects as arguments. This approach can be useful for operations involving two objects.

Algorithm:

1. Start the program.
2. Define the `Complex` class with real and imaginary data members and declare `add()` as a friend.
3. Initialise two complex-number objects.
4. Pass both objects to the friend function.
5. Add their real components and imaginary components separately.
6. Display the resulting complex number and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Constructor to initialize data members
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Friend function declaration
    friend Complex add(Complex, Complex);
    void display()
    {
        cout << real << " + " << imag << "i";
    }
};
// Friend function definition
Complex add(Complex c1, Complex c2)
{
```

```
    Complex c3;
    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;
    return c3;
}
int main()
{
    float r1, i1, r2, i2;
    cout << "Enter real and imaginary part of first complex number: ";
    cin >> r1 >> i1;
    cout << "Enter real and imaginary part of second complex number: ";
    cin >> r2 >> i2;
    Complex c1(r1, i1);
    Complex c2(r2, i2);
    Complex c3 = add(c1, c2);   // Addition using friend function
    cout << "\nFirst Complex Number  = ";
    c1.display();
    cout << "\nSecond Complex Number = ";
    c2.display();
    cout << "\nAddition              = ";
    c3.display();
    return 0;
}
```

Output:

```
Enter real and imaginary part of first complex number: 7 4
Enter real and imaginary part of second complex number: 2 6

First Complex Number  = 7 + 4i
Second Complex Number = 2 + 6i
Addition              = 9 + 10i
```

Conclusion:

The sum of two complex numbers was obtained using a friend function `add()`, which is not a member of the class but could still access its private data members `real` and `imag`. The numbers 5 + 3i and 4 + 2i gave the result 9 + 5i, and a constructor was used to initialise the objects. The practical showed how friend functions can be used when an operation involves objects of a class but does not naturally belong to any one object.

Program – 23

Aim:

To write a C++ program to find the mean of the given numbers using a friend function.

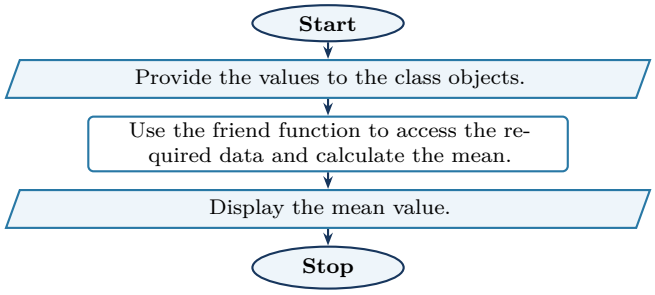
Theory:

A friend function can access private members of a class even though it is defined outside the class. In this program, it receives an object containing an array, traverses the stored values, and divides their sum by the number of values to calculate the arithmetic mean.

Algorithm:

1. Start the program.
2. Define the `Mean` class with an array and declare `calculateMean()` as a friend.
3. Read the values into the object's array.
4. Pass the object to the friend function.
5. Add the values and divide the sum by the number of elements.
6. Display the mean and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Mean
{
    int n;
    float arr[100];
public:
    Mean(int count)
    {
        n = count;
    }
    void getData()
    {
        cout << "Enter " << n << " numbers: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }
    friend float calculateMean(Mean);
};
// Friend function
float calculateMean(Mean m)
{
```

```
    float sum = 0;
    for (int i = 0; i < m.n; i++)
        sum += m.arr[i];
    return sum / m.n;
}
int main()
{
    int n;
    cout << "Enter number of values: ";
    cin >> n;
    Mean m(n);
    m.getData();
    cout << "Mean = " << calculateMean(m);
    return 0;
}
```

Output:

```
Enter number of values: 4
Enter 4 numbers: 12 18 24 30
Mean = 21
```

Conclusion:

A friend function `calculateMean()` was used to access the private array of the `Mean` class and calculate the average of the entered values; for 10, 20, 30, 40 and 50 the mean was 30. The practical showed that a friend function can read the private data of an object passed to it, and combined this with loops and arrays to perform a simple statistical calculation.

Program – 24

Aim:

To write a C++ program to add the integer values of two class objects.

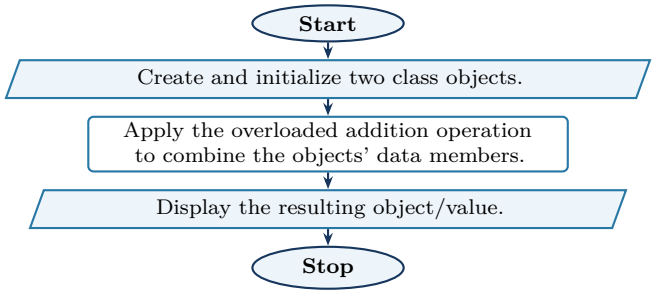
Theory:

Operator overloading gives an existing C++ operator a suitable meaning for a user-defined type. A binary operator implemented as a member function receives the right-hand operand as its argument, while the calling object supplies the left-hand operand. This permits object addition to use the familiar `+` notation.

Algorithm:

1. Start the program.
2. Define the `Number` class and overload the binary `+` operator.
3. Create two objects and initialise their values.
4. Add the objects using the overloaded operator and store the returned object.
5. Display the value of the result object.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    // Constructor
    Number(int v = 0)
    {
        value = v;
    }
    // Operator overloading for +
    Number operator+(Number obj)
    {
        Number temp;
        temp.value = value + obj.value;
        return temp;
    }
    void display()
    {
        cout << "Sum = " << value << endl;
```

```
    }
};
int main()
{
    Number n1(10), n2(20), n3;
    n3 = n1 + n2;
    n3.display();
    return 0;
}
```

Output:

```
Sum = 30
```

Conclusion:

The binary `+` operator was overloaded for the class `Number` using the `operator` keyword so that two objects could be added with the natural expression `n3 = n1 + n2`. The objects holding 10 and 20 produced a sum of 30. This practical showed that operator overloading makes user-defined types behave like built-in types and improves the readability of the code, without changing the meaning of the operator for ordinary types.

Program – 25

Aim:

To write a C++ program for the addition of two time objects.

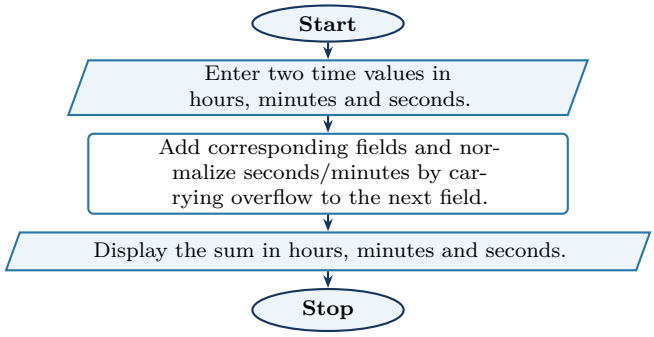
Theory:

Operator overloading allows the `+` operator to add two objects of a `Time` class. Because seconds and minutes have fixed ranges, the result must be normalised: each 60 seconds contributes one minute, and each 60 minutes contributes one hour. The quotient and remainder operators support this conversion.

Algorithm:

1. Start the program.
2. Define the `Time` class and overload `+` for two time objects.
3. Initialise the two time values.
4. Add their hours, minutes and seconds.
5. Convert every 60 seconds into one minute and every 60 minutes into one hour, carrying excess units forward.
6. Display the normalised sum and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Time
{
    int hours, minutes, seconds;
public:
    void getTime()
    {
        cout << "Enter hours, minutes and seconds: ";
        cin >> hours >> minutes >> seconds;
    }
    void display()
    {
        cout << hours << " hours "
             << minutes << " minutes "
             << seconds << " seconds";
    }
    // Operator overloading for +
```

```
    Time operator+(Time t)
    {
        Time temp;
        temp.seconds = seconds + t.seconds;
        temp.minutes = minutes + t.minutes + temp.seconds / 60;
        temp.seconds = temp.seconds % 60;
        temp.hours   = hours + t.hours + temp.minutes / 60;
        temp.minutes = temp.minutes % 60;
        return temp;
    }
};
int main()
{
    Time t1, t2, t3;
    cout << "Enter first time:\n";
    t1.getTime();
    cout << "Enter second time:\n";
    t2.getTime();
    t3 = t1 + t2;
    cout << "\nFirst Time  = ";
    t1.display();
    cout << "\nSecond Time = ";
    t2.display();
    cout << "\nAddition of Two Times = ";
    t3.display();
    return 0;
}
```

Output:

```
Enter first time:
Enter hours, minutes and seconds: 2 50 45
Enter second time:
Enter hours, minutes and seconds: 4 25 35

First Time  = 2 hours 50 minutes 45 seconds
Second Time = 4 hours 25 minutes 35 seconds
Addition of Two Times = 7 hours 16 minutes 20 seconds
```

Conclusion:

Two `Time` objects were added by overloading the `+` operator, with the carry from seconds to minutes and from minutes to hours handled using the `/` and `%` operators. The times 5 h 45 min 50 s and 3 h 25 min 30 s gave a total of 9 hours 11 minutes 20 seconds. The practical showed how operator overloading can include extra logic so that the result of the operation follows the rules of the problem domain.

Program – 26

Aim:

To write a C++ program to design a class having a static member function `showcount()` that displays the number of objects created.

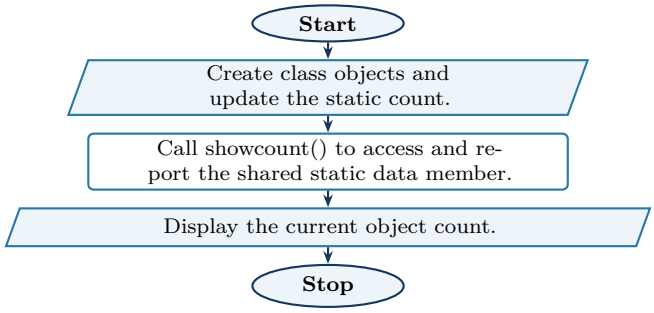
Theory:

A static member function belongs to the class rather than to an individual object and can be called using the class name. It does not have a `this` pointer and can directly access only static data members. A static counter can therefore track the number of objects created.

Algorithm:

1. Start the program.
2. Declare a static data member to store the object count.
3. Increment the counter in the constructor.
4. Create four objects of the class.
5. Call `showcount()` through the class name and display the count.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Demo
{
    static int count;
public:
    Demo()
    {
        count++;
    }
    static void showcount()
    {
        cout << "Number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Demo::count = 0;
int main()
{
    Demo d1;
```

```
    Demo d2;
    Demo d3;
    Demo d4;
    Demo::showcount();
    return 0;
}
```

Output:

```
Number of objects created = 4
```

Conclusion:

A static member function `showcount()` was called with the class name and displayed the number of objects created; after four objects were created the output was 4. Because a static function can only use static data, it works on the shared counter and does not need an object. The practical reinforced the difference between per-object data and class-level data in C++.

Program – 27

Aim:

To write a C++ program that uses unary operator (`++` / `--`) overloading.

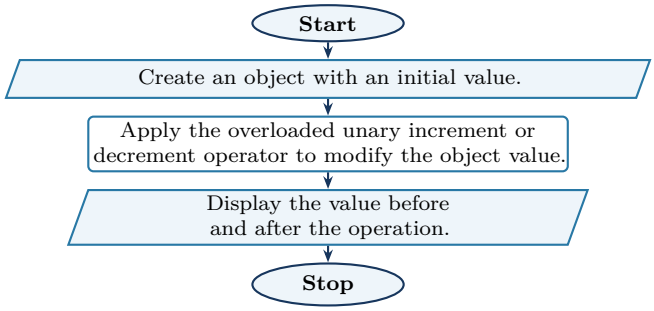
Theory:

Unary operator overloading defines how an operator such as `++` or `-` acts on an object. When overloaded as a member function, a unary operator takes no explicit operand argument because the calling object is the operand. The function changes the object's stored number.

Algorithm:

1. Start the program.
2. Define a `Number` class and overload the unary increment and decrement operators.
3. Create an object with an initial value.
4. Apply `++` and display the updated value.
5. Apply `-` as shown in the program and display the updated value.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    Number(int v)
    {
        value = v;
    }
    // Unary ++ operator overloading
    void operator++()
    {
        ++value;
    }
    // Unary -- operator overloading
    void operator--()
    {
        --value;
    }
    void display()
```

```
    {
        cout << "Value = " << value << endl;
    }
};
int main()
{
    Number n(10);
    cout << "Before increment:" << endl;
    n.display();
    ++n;
    cout << "After increment:" << endl;
    n.display();
    --n;
    --n;
    cout << "After two decrements:" << endl;
    n.display();
    return 0;
}
```

Output:

```
Before increment:
Value = 24
After increment:
Value = 25
After two decrements:
Value = 23
```

Conclusion:

The unary operators `++` and `-` were overloaded as member functions of the class `Number`. The object holding 10 became 11 after `++` and then 9 after two applications of `-`. The practical showed that unary operator overloading takes no explicit argument, since the operand is the object itself, and that overloaded operators can modify the data members of an object just like the built-in operators modify ordinary variables.

Program – 28

Aim:

To write a C++ program that uses binary operator overloading on strings.

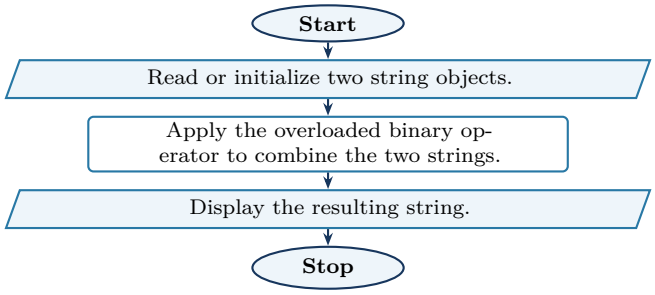
Theory:

Binary operator overloading lets the `+` operator combine two objects of a user-defined string class. The program uses `strcpy()` to copy the first character array and `strcat()` to append the second. These C-style string functions require enough space in the destination array for the combined text and its null terminator.

Algorithm:

1. Start the program.
2. Define the `String` class and overload the binary `+` operator.
3. Initialise two string objects.
4. Copy the first string into the result buffer using `strcpy()`.
5. Append the second string using `strcat()`.
6. Display the concatenated result and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <cstring>
using namespace std;
class String
{
    char str[100];
public:
    // Constructor
    String(const char s[] = "")
    {
        strcpy(str, s);
    }
    // Binary + operator overloading (concatenation)
    String operator+(String s)
    {
        String temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }
    void display()
```

```
    {
        cout << str << endl;
    }
};
int main()
{
    String s1("Hello ");
    String s2("World");
    String s3;
    s3 = s1 + s2;
    cout << "First String        : ";
    s1.display();
    cout << "Second String       : ";
    s2.display();
    cout << "Concatenated String : ";
    s3.display();
    return 0;
}
```

Output:

```
First String        : Digital
Second String       : Logic
Concatenated String : Digital Logic
```

Conclusion:

The `+` operator was overloaded for a `String` class to concatenate two strings using `strcpy()` and `strcat()`, so that the strings "Hello " and "World" produced "Hello World". This practical demonstrated binary operator overloading on a non-numeric type and showed how an operator can be given a meaning that suits the class, in the same way that `+` joins strings in many other languages.

Program – 29

Aim:

To write a C++ program to create a Matrix class and overload the + operator to add two matrix objects.

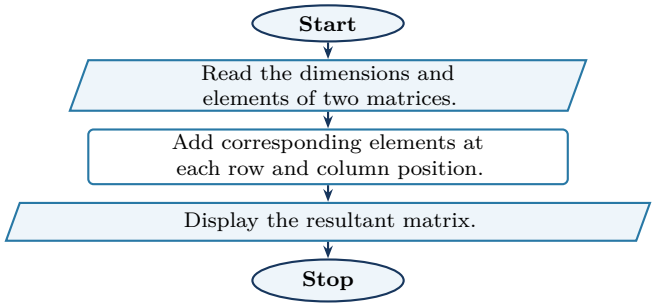
Theory:

A two-dimensional array represents a matrix using rows and columns. Matrix addition is performed by adding elements at matching row and column positions, and the matrices must have the same dimensions. Nested loops visit each position and store the sum in the result matrix.

Algorithm:

1. Start the program.
2. Define the matrix class and overload the + operator.
3. Read or initialise the elements of both matrices.
4. Use nested loops to add corresponding elements into the result matrix.
5. Display the resultant matrix row by row.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
class Matrix
{
    int a[2][2];
public:
    // Input matrix elements
    void getdata()
    {
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                cin >> a[i][j];
    }
    // Binary + operator overloading
    Matrix operator+(Matrix m)
    {
        Matrix temp;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                temp.a[i][j] = a[i][j] + m.a[i][j];
```

```
        return temp;
    }
    // Display matrix
    void display()
    {
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
                cout << a[i][j] << "\t";
            cout << endl;
        }
    }
};
int main()
{
    Matrix m1, m2, m3;
    cout << "Enter elements of first matrix:" << endl;
    m1.getdata();
    cout << "Enter elements of second matrix:" << endl;
    m2.getdata();
    m3 = m1 + m2;     // Add two matrix objects
    cout << "\nFirst Matrix:" << endl;
    m1.display();
    cout << "\nSecond Matrix:" << endl;
    m2.display();
    cout << "\nResultant Matrix:" << endl;
    m3.display();
    return 0;
}
```

Output:

```
Enter elements of first matrix:
2 4
6 8
Enter elements of second matrix:
1 3
5 7

First Matrix:
2   4
6   8

Second Matrix:
1   3
5   7

Resultant Matrix:
3   7
11  15
```

Conclusion:

Two 2 x 2 matrix objects were added by overloading the + operator, which adds the corresponding elements using nested loops. Adding the matrices [2 4; 6 8] and [1 3; 5 7] gave the resultant matrix [3 7; 11 15]. The practical combined two-dimensional arrays, loops and operator overloading, and showed how a mathematical operation on a complex data structure can be written as a single natural expression.

Program – 30

Aim:

To write a C++ program with two classes, `Polar` and `Rectangle`, that represent a point in the polar and rectangular systems and convert from one system to the other.

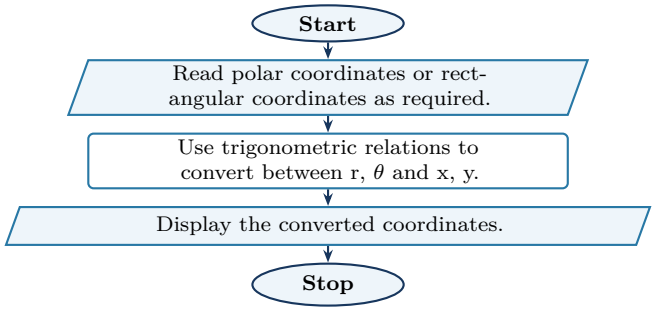
Theory:

Cartesian (rectangular) coordinates describe a point by x, y, while polar coordinates use radius r and angle θ. The conversions are $r = \sqrt{x^2 + y^2}$, $\theta = \text{atan2}(y, x)$, $x = r\cos\theta$ and $y = r\sin\theta$. Conversion constructors can initialise one class from an object of the other type.

Algorithm:

1. Start the program.
2. Define the rectangular and polar coordinate classes and their conversion constructors.
3. Create a rectangular point with the input coordinates.
4. Convert the point to polar form using radius and angle calculations.
5. Convert the polar object back to rectangular form.
6. Display both representations and stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;
class Polar;
class Rectangle
{
    float x, y;
public:
    Rectangle(float a = 0, float b = 0)
    {
        x = a;
        y = b;
    }
    Rectangle(Polar p);   // Conversion constructor (Polar -> Rectangle)
    friend class Polar;
    void display()
    {
        cout << "Rectangular Coordinates = (" << x << ", " << y << ")" << endl;
    }
};
```

```
class Polar
{
    float r, theta;
public:
    Polar(float radius = 0, float angle = 0)
    {
        r = radius;
        theta = angle;
    }
    Polar(Rectangle rec)  // Conversion constructor (Rectangle -> Polar)
    {
        r = sqrt(rec.x * rec.x + rec.y * rec.y);
        theta = atan2(rec.y, rec.x) * 180 / 3.14159;
    }
    friend class Rectangle;
    void display()
    {
        cout << "Polar Coordinates = (" << r << ", " << theta << " degrees)" << endl;
    }
};
Rectangle::Rectangle(Polar p)
{
    float angle = p.theta * 3.14159 / 180;
    x = p.r * cos(angle);
    y = p.r * sin(angle);
}
int main()
{
    Rectangle R1(3, 4);
    cout << "Original Rectangle:\n";
    R1.display();
    Polar P1 = R1;            // Rectangle -> Polar
    cout << "\nRectangle to Polar:\n";
    P1.display();
    Rectangle R2 = P1;        // Polar -> Rectangle
    cout << "\nPolar to Rectangle:\n";
    R2.display();
    return 0;
}
```

Output:

```
Original Rectangle:
Rectangular Coordinates = (3, 4)

Rectangle to Polar:
Polar Coordinates = (5, 53.1301 degrees)

Polar to Rectangle:
Rectangular Coordinates = (3, 4)
```

Conclusion:

Two classes, `Rectangle` and `Polar`, were created and conversion constructors were used to convert a point from one system to the other. The point (3, 4) was converted to polar form (5, 53.1301 degrees) and back to (3, 4), confirming that both conversions are consistent. The practical showed how a user-defined type conversion can be written, how friend classes allow two classes to access each other's private data, and how the `<cmath>` functions `sqrt()`, `atan2()`, `cos()` and `sin()` are used for coordinate conversion.

Program – 31

Aim:

To write a C++ program to implement single inheritance.

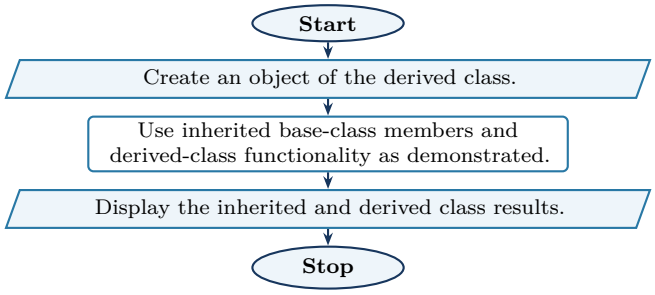
Theory:

Inheritance lets a derived class reuse accessible members of a base class. In single inheritance, one derived class inherits from one base class. A `protected` member is not publicly accessible from outside the class but can be used by the class itself and its derived classes.

Algorithm:

1. Start the program.
2. Define the base class `Student` with student details and related functions.
3. Derive the `Result` class from `Student` and add the marks member.
4. Create an object of the derived class.
5. Read and display the inherited student details along with the marks.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
// Base Class
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
    void displayStudentData()
    {
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
```

```
};
// Derived Class
class Result : public Student
{
private:
    float marks;
public:
    void getResult()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
    void displayResult()
    {
        displayStudentData();
        cout << "Marks: " << marks << endl;
    }
};
int main()
{
    Result r;
    r.getStudentData();
    r.getResult();
    cout << "\n--- Student Result ---\n";
    r.displayResult();
    return 0;
}
```

Output:

```
Enter Roll Number: 217
Enter Name: Aditi Shah
Enter Marks: 92

--- Student Result ---
Roll Number: 217
Name: Aditi Shah
Marks: 92
```

Conclusion:

The derived class `Result` inherited the data members and functions of the base class `Student` and added its own member `marks`, so that one object could read and display both the student's details and the marks. The practical demonstrated single inheritance, the use of the `protected` access specifier so that derived classes can use base class data, and the advantage of code reuse, since the base class code did not have to be rewritten.

Program – 32

Aim:

To write a C++ program to implement multiple inheritance.

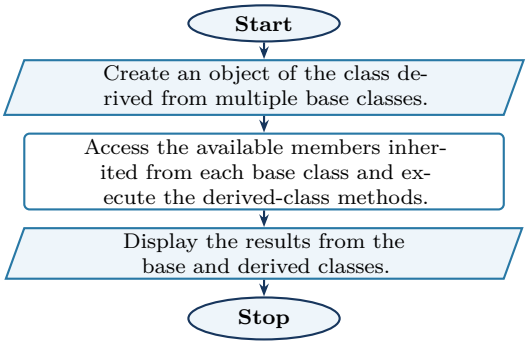
Theory:

Multiple inheritance allows a derived class to inherit members from more than one base class. Each base class can encapsulate a distinct group of information, while the derived class combines those features into a single object. Access to inherited members depends on the access specifiers and inheritance mode.

Algorithm:

1. Start the program.
2. Define the `Student` base class for roll number and name.
3. Define the `Marks` base class for marks information.
4. Derive `Result` from both base classes.
5. Create a `Result` object and read/display the inherited information.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
// Base Class 1
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
};
// Base Class 2
```

```
class Marks
{
protected:
    float marks;
public:
    void getMarks()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
};
// Derived Class
class Result : public Student, public Marks
{
public:
    void display()
    {
        cout << "\n--- Student Result ---" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};
int main()
{
    Result r;
    r.getStudentData();
    r.getMarks();
    r.display();
    return 0;
}
```

Output:

```
Enter Roll Number: 308
Enter Name: Krish Patel
Enter Marks: 78

--- Student Result ---
Roll Number: 308
Name: Krish Patel
Marks: 78
```

Conclusion:

The class `Result` was derived from two base classes, `Student` and `Marks`, and an object of it could use the members of both classes to read and display the roll number, name and marks. The practical demonstrated multiple inheritance and showed that a derived class can combine the features of several independent base classes, which supports code reuse while keeping each base class focused on one responsibility.

Program – 33

Aim:

To write a C++ program to create a file using a constructor function.

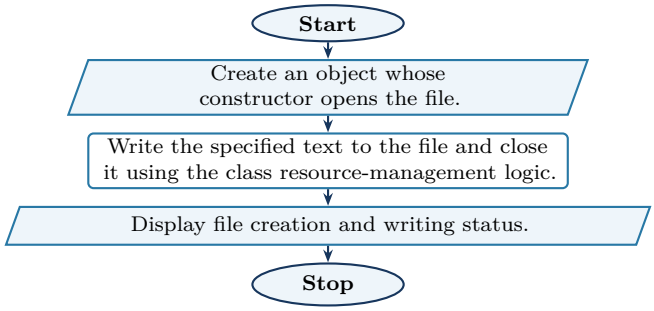
Theory:

The `ofstream` class from `<fstream>` supports writing data to a file. Opening a file in a constructor can tie the file's lifetime to the object, while the destructor can close it automatically when the object is destroyed. The program checks whether the file opened successfully before writing.

Algorithm:

1. Start the program.
2. Define `FileHandler` with an `ofstream` member.
3. Open the named file in the constructor and report whether opening succeeded.
4. Call the write function to write the two text lines if the file is open.
5. When the object is destroyed, close the open file in the destructor.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
class FileHandler
{
    ofstream file;
public:
    // Constructor creates (opens) the file
    FileHandler(const string &filename)
    {
        file.open(filename);
        if (file.is_open())
            cout << "File \"" << filename << "\" created successfully." << endl;
        else
            cout << "Error: unable to create file." << endl;
    }
    void write(const string &text)
    {
        if (file.is_open())
```

```
            file << text << endl;
    }
    // Destructor closes the file
    ~FileHandler()
    {
        if (file.is_open())
        {
            file.close();
            cout << "File closed." << endl;
        }
    }
};
int main()
{
    FileHandler f("sample.txt");
    f.write("This file was created using a constructor.");
    f.write("Object Oriented Programming Lab.");
    cout << "Data written to file." << endl;
    return 0;
}
```

Output:

```
File "sample.txt" created successfully.
Data written to file.
File closed.
```

Conclusion:

A file was created automatically when an object of the class `FileHandler` was constructed, since the constructor opened the file using an `ofstream` object. Two lines of text were written to it, and the destructor closed it when the object went out of scope, as shown by the messages displayed. The practical showed how the constructor and destructor can manage a resource, such as a file, so that it is always opened and closed properly, and introduced the file handling classes of the `<fstream>` header.

Program – 34

Aim:

To write a C++ program to demonstrate the use of templates.

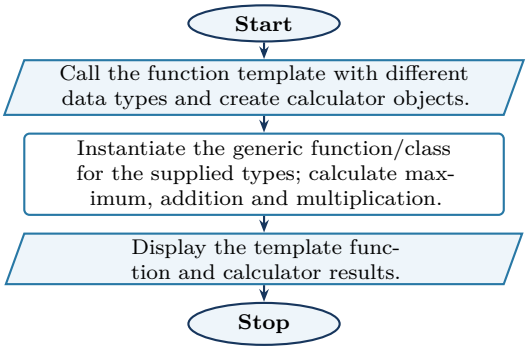
Theory:

Templates support generic programming by allowing functions and classes to operate on a placeholder type. The compiler determines or is given the type when the template is used, enabling the same definition to work with types such as `int`, `double`, `char` and `float`. A function template and a class template are demonstrated separately.

Algorithm:

1. Start the program.
2. Define the function template `maximum()` to return the larger of two values.
3. Call the function template with integer, double and character arguments.
4. Define the class template `Calculator<T>` with addition and multiplication functions.
5. Create integer and floating-point calculator objects and display their results.
6. Stop.

Flowchart:



C++ Code:

```cpp
#include <iostream>
using namespace std;
// Function template
template <class T>
T maximum(T a, T b)
{
    return (a > b) ? a : b;
}
// Class template
template <class T>
class Calculator
{
    T a, b;
public:
    Calculator(T x, T y)
    {
        a = x;
        b = y;
    }
```

```
    T add()      { return a + b; }
    T multiply() { return a * b; }
};
int main()
{
    cout << "Max of 10 and 25      = " << maximum(10, 25) << endl;
    cout << "Max of 3.5 and 2.1    = " << maximum(3.5, 2.1) << endl;
    cout << "Max of 'a' and 'z'    = " << maximum('a', 'z') << endl;
    Calculator<int> c1(6, 4);
    cout << "\nInteger Calculator:" << endl;
    cout << "Sum     = " << c1.add() << endl;
    cout << "Product = " << c1.multiply() << endl;
    Calculator<float> c2(2.5, 1.5);
    cout << "\nFloat Calculator:" << endl;
    cout << "Sum     = " << c2.add() << endl;
    cout << "Product = " << c2.multiply() << endl;
    return 0;
}
```

Output:

```
Max of 10 and 25      = 25
Max of 3.5 and 2.1    = 3.5
Max of 'a' and 'z'    = z

Integer Calculator:
Sum     = 10
Product = 24

Float Calculator:
Sum     = 4
Product = 3.75
```

Conclusion:

A function template `maximum()` returned the larger of two values for `int`, `double` and `char` arguments (25, 3.5 and z), and a class template `Calculator` produced correct results for both integer (sum 10, product 24) and floating-point (sum 4, product 3.75) data. The practical showed that a single template definition replaces several nearly identical functions or classes, and that the compiler generates the code for each data type used, which gives type-safe, reusable and generic programs.